1. Ejercicio

Enunciado

Estudia el siguiente diagrama. Fíjate en los siguientes aspectos:

a. La visibilidad de los atributos y funciones
b. La multiplicidad de las asociaciones
c. Qué tipo de relaciones hay entre las clases ¿hay alguna jerarquía?
d. ¿Cómo crees que se representarán esas relaciones al convertir el diagrama en código?

Respuesta

a) Visibilidad

En UML se usan estos símbolos:

- `+` público
- `-` privado
- `#` protegido

Normalmente:

- **Atributos → privados (-)**
- **Métodos → públicos (+)**

Esto significa que los atributos solo se modifican mediante métodos.

---

b) Multiplicidad

Indica cuántos objetos pueden relacionarse.

Ejemplos típicos:

- `1` → uno exactamente
- `0..1` → opcional

- `1..*` → uno o muchos
- `*` → muchos

Ejemplo del diagrama:
Un **Profesor** puede tener **muchos alumnos**, pero cada alumno pertenece a **un profesor**.

---

c) Tipo de relaciones

En el diagrama aparecen:

- **Asociación** (línea simple entre clases)
- **Herencia / generalización** (flecha con triángulo)

Por ejemplo:

Persona
↑
Alumno
Profesor

Esto significa que **Alumno y Profesor heredan de Persona**.

---

d) Representación en código

Se traducen así:

- **Herencia**

```
class Alumno extends Persona { }
class Profesor extends Persona { }
```

- **Asociación**

```
class Profesor {
  List<Alumno> alumnos;
}
```

---

2. Ejercicio

Enunciado

Ahora vamos con otro diagrama. Fíjate en los siguientes aspectos:

a. La visibilidad de los atributos y funciones
b. La multiplicidad de las asociaciones
c. Qué tipo de relaciones hay entre las clases ¿hay alguna jerarquía?
d. ¿Cómo crees que se representarán esas relaciones al convertir el diagrama en código?
e. No aparecen los métodos explicitados en las clases. Imagina cuáles sería interesante incluir.

Respuesta

a) Visibilidad

Lo correcto sería:

- atributos → `-`
- métodos → `+`

Para aplicar **encapsulación**.

---

b) Multiplicidad

Por ejemplo:

- Un objeto puede tener **muchos asociados**
- Otro solo puede tener **uno**

En código suele traducirse en:

- un objeto → variable simple
- muchos objetos → `List`, `ArrayList`, etc.

---

c) Relaciones

En el diagrama hay:

- **asociaciones**
- posiblemente **herencia**

Las asociaciones indican que una clase **usa o contiene** otra.

---

d) En código

Ejemplo:

```
class Pedido {
   Cliente cliente;
   List<Producto> productos;
}
```

---

e) Métodos que podrían añadirse

Depende de las clases, pero normalmente:

Ejemplo para una clase **Pedido**

```
añadirProducto()
eliminarProducto()
calcularTotal()
```

Ejemplo para **Cliente**

```
realizarPedido()
consultarPedidos()
```

---

3. Ejercicio

Enunciado

Dado el siguiente diagrama de clases:

a. Retoca las relaciones entre clases ¿Debería ser alguna una composición o agregación?
b. Reflexiona sobre cuáles serían las multiplicidades en las relaciones en las que tenga sentido ponerlas

c. Qué tipo de relaciones hay entre las clases ¿hay alguna jerarquía?
d. ¿Cómo crees que se representarán esas relaciones al convertir el diagrama en código?

Respuesta

a) Composición o agregación

- **Composición** → el objeto depende totalmente del otro
- **Agregación** → relación más débil

Ejemplo correcto:

Pedido – LíneaPedido

→ **composición**

Porque si se borra el pedido se borran sus líneas.

Pedido ♦—— LíneaPedido

---

b) Multiplicidades

Ejemplo típico:

Cliente 1 ----- * Pedido
Pedido 1 ----- * LineaPedido
Producto 1 ---- * LineaPedido

Interpretación:

- un cliente puede tener muchos pedidos
- un pedido tiene muchas líneas
- cada línea pertenece a un producto

---

c) Jerarquía

Puede existir **herencia**, por ejemplo:

Empleado
↑

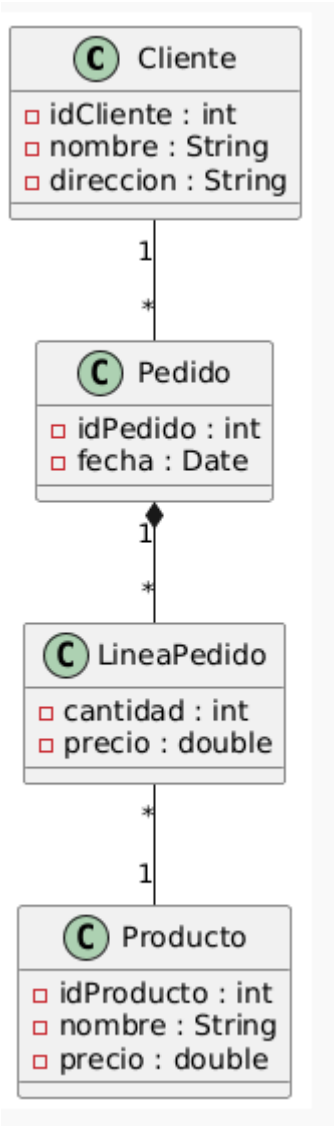
Gerente
Vendedor

---

d) Representación en código

```
class Pedido {
   Cliente cliente;
   List<LineaPedido> lineas;
}

class LineaPedido {
   Producto producto;
   int cantidad;
}
```



---

4. Ejercicio

Enunciado

Dado el siguiente diagrama de clases:

a. Observa las relaciones de agregación y composición e indica en qué influye en el código posterior que sean de un tipo o de otro
b. Revisa las multiplicidades y anota en qué clases aparecerán “colecciones” de objetos de otra clase al codificar
c. Observa la visibilidad de atributos y métodos ¿Cambiarías algo?

Respuesta

a) Influencia en el código

- **Composición**
    - el objeto depende del otro
    - normalmente se crea dentro de la clase

Ejemplo:

```
class Pedido {
  List<LineaPedido> lineas = new ArrayList<>();
}
```

- **Agregación**
    - el objeto puede existir independientemente

Ejemplo:

```
class Equipo {
  List<Jugador> jugadores;
}
```

Los jugadores existen aunque el equipo desaparezca.

---

b) Clases que tendrán colecciones

Donde haya multiplicidad *.

Ejemplo:

Pedido 1 ----- * LineaPedido

Entonces en código:

List<LineaPedido> lineas;

---

c) Visibilidad

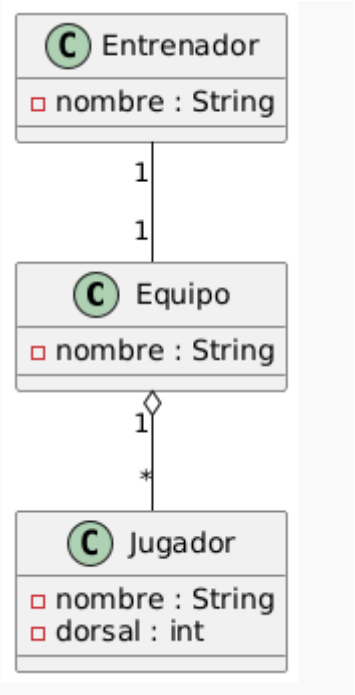
Lo correcto sería:

- atributos → **privados**
- métodos → **públicos**

Ejemplo:

private String nombre;
public String getNombre()

Esto sigue el principio de **encapsulación**.



---

5. Ejercicio

Enunciado

Dado el siguiente diagrama de clases, cómo se podría modificar para:

- Que docentes y estudiantes optimicen su estructura
- Que un docente sea tutor de otro docente en prácticas
- Que una misma asignatura pudiera estar en más de un programa
- Que una materia docente la pueda impartir más de un profesor

---

Respuesta

Optimizar docentes y estudiantes

Crear una **superclase común**:

```
Persona
├── Estudiante
└── Docente
```

Así se comparten atributos como:

- nombre
- DNI
- dirección
- email

Docente tutor de otro docente

Relación recursiva:

```
Docente
  │
  └── tutor → Docente
```

Multiplicidad:

Docente (tutor) 1 ----- * Docente (prácticas)

Una asignatura en varios programas

Relación **muchos a muchos**:

Programa * ----- * Asignatura

En código se resuelve con una tabla intermedia.

Una materia impartida por varios profesores

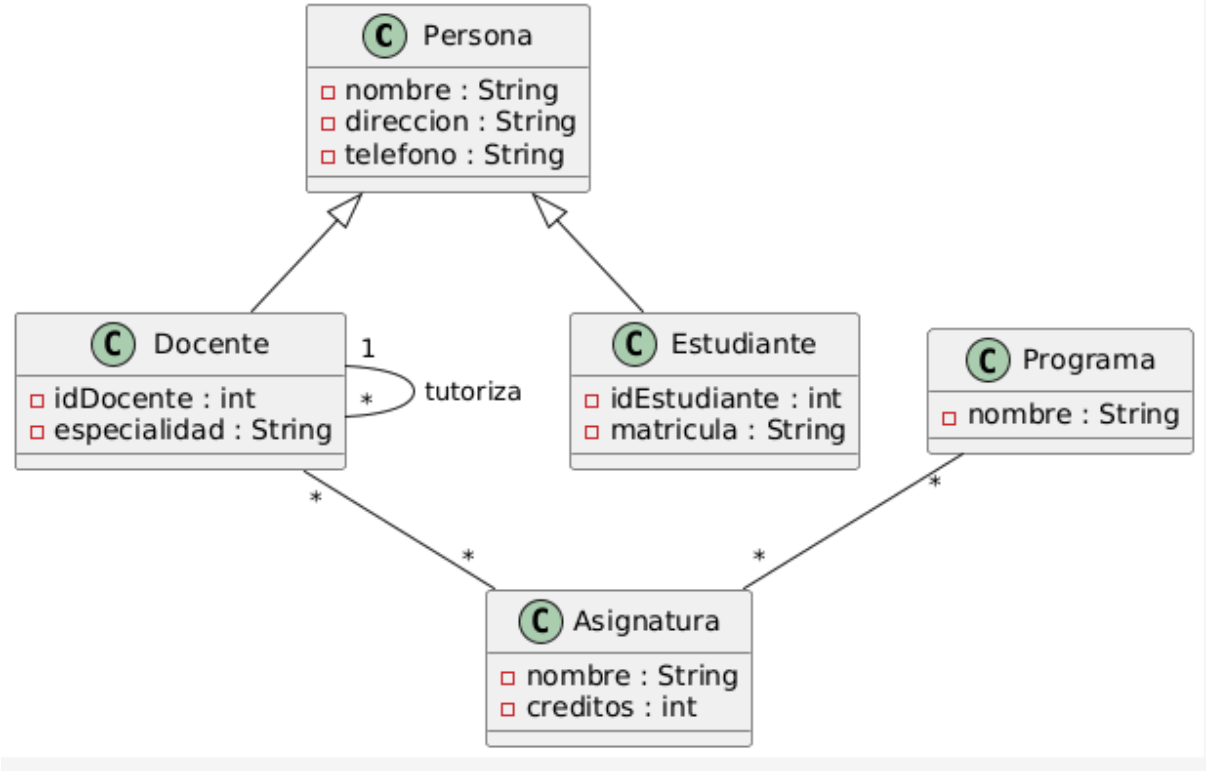
Otra relación **muchos a muchos**:

Docente * ----- * Asignatura

En código:

```
class Docente {
  List<Asignatura> asignaturas;
}

class Asignatura {
  List<Docente> profesores;
}
```



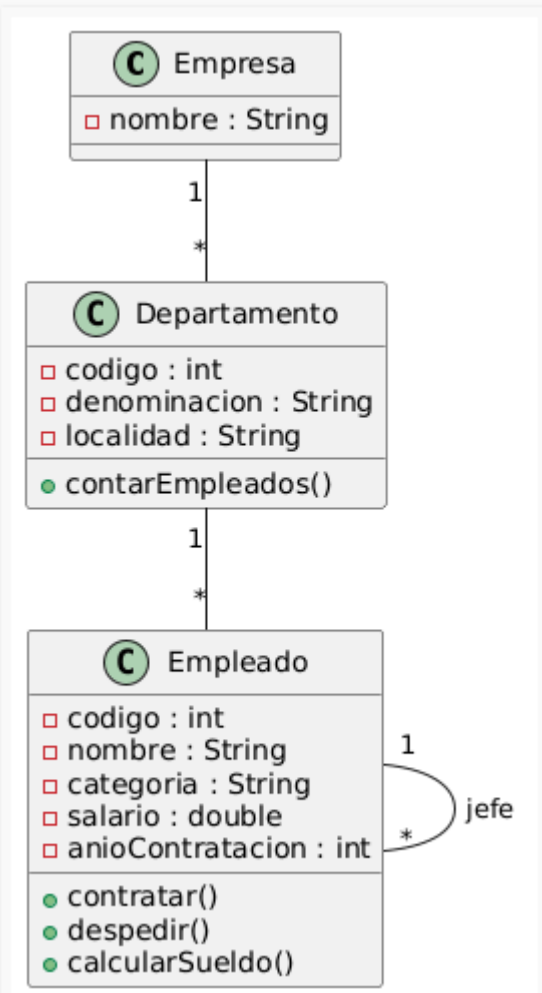
Ejercicio 6 – Empresa, departamentos y empleados

Clases necesarias

- Empresa
- Departamento
- Empleado

Relaciones importantes

- Una **empresa tiene varios departamentos**
- Un **departamento tiene muchos empleados**
- Un **empleado puede ser jefe de varios empleados**
- Un **empleado solo tiene un jefe**


Empresa
nombre : String
1
*
Departamento
codigo : int
denominacion : String
localidad : String
contarEmpleados()
1
*
Empleado
codigo : int
nombre : String
categoria : String
salario : double
anioContratacion : int
contratar()
despedir()
calcularSueldo()
1
jefe
*

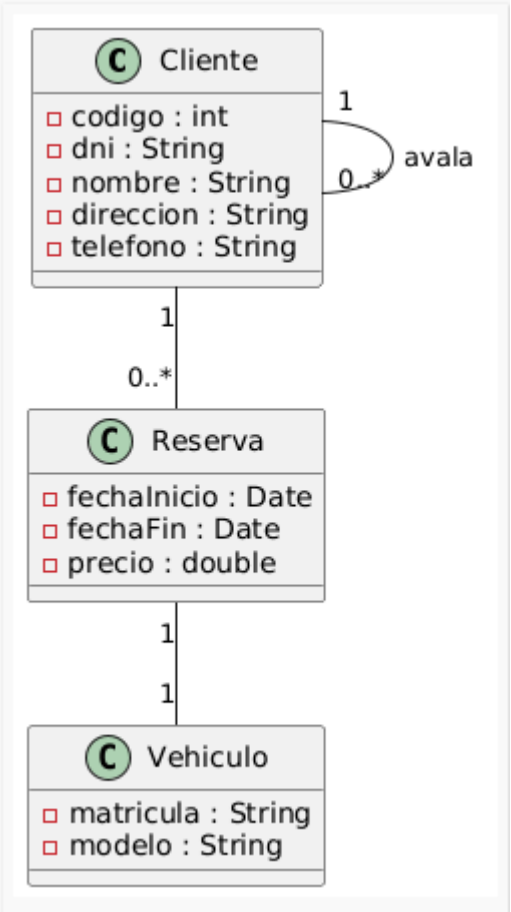
Ejercicio 7 – Sistema de alquiler de vehículos

Clases

- Cliente
- Reserva
- Vehiculo

Reglas

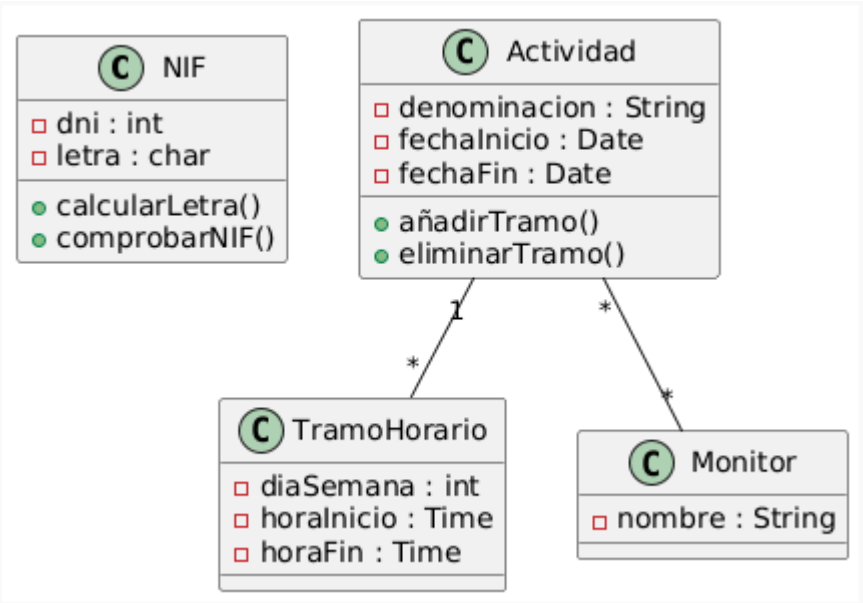
- Un cliente puede tener **0..*** reservas
- Una reserva es de **un cliente**
- Una reserva usa **un vehículo**
- Un cliente puede **avalar a otro**



Ejercicio 8 – Actividades y monitores

Clases

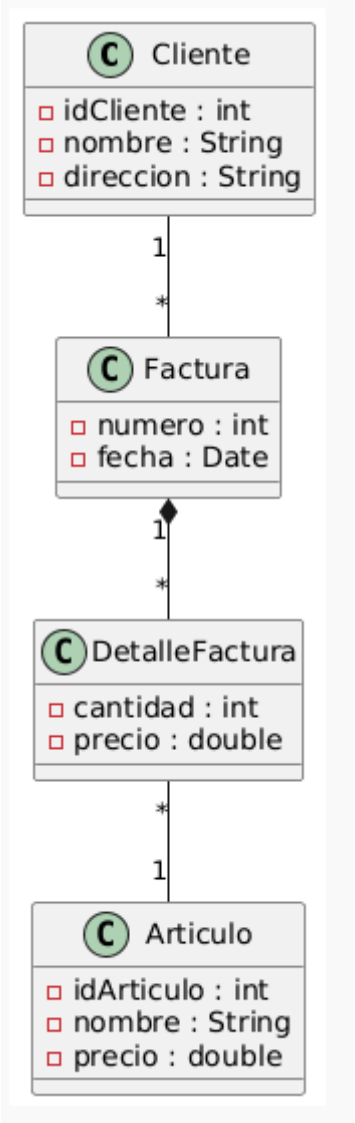
- NIF
- Actividad
- TramoHorario
- Monitor



Ejercicio 9 – Sistema de ventas (almacén)

Clases

- Cliente
- Articulo
- Factura
- DetalleFactura



Ejercicio 10 – Sistema con clases Entity, Control y Boundary

Tipos de clases

Entidad

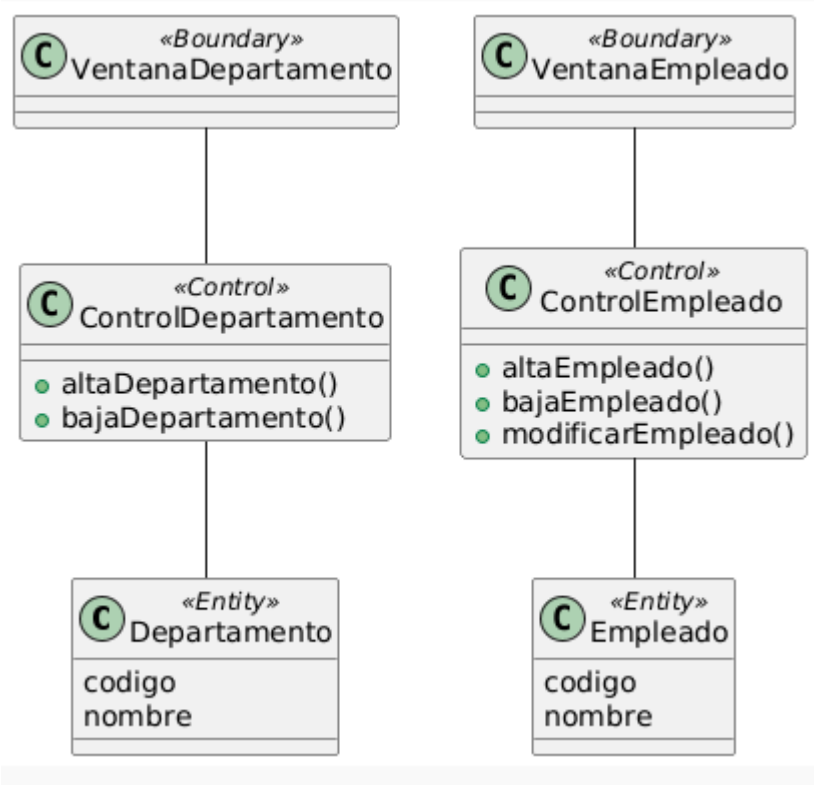
- Departamento
- Empleado

Control

- ControlEmpleado
- ControlDepartamento

Interfaz (Boundary)

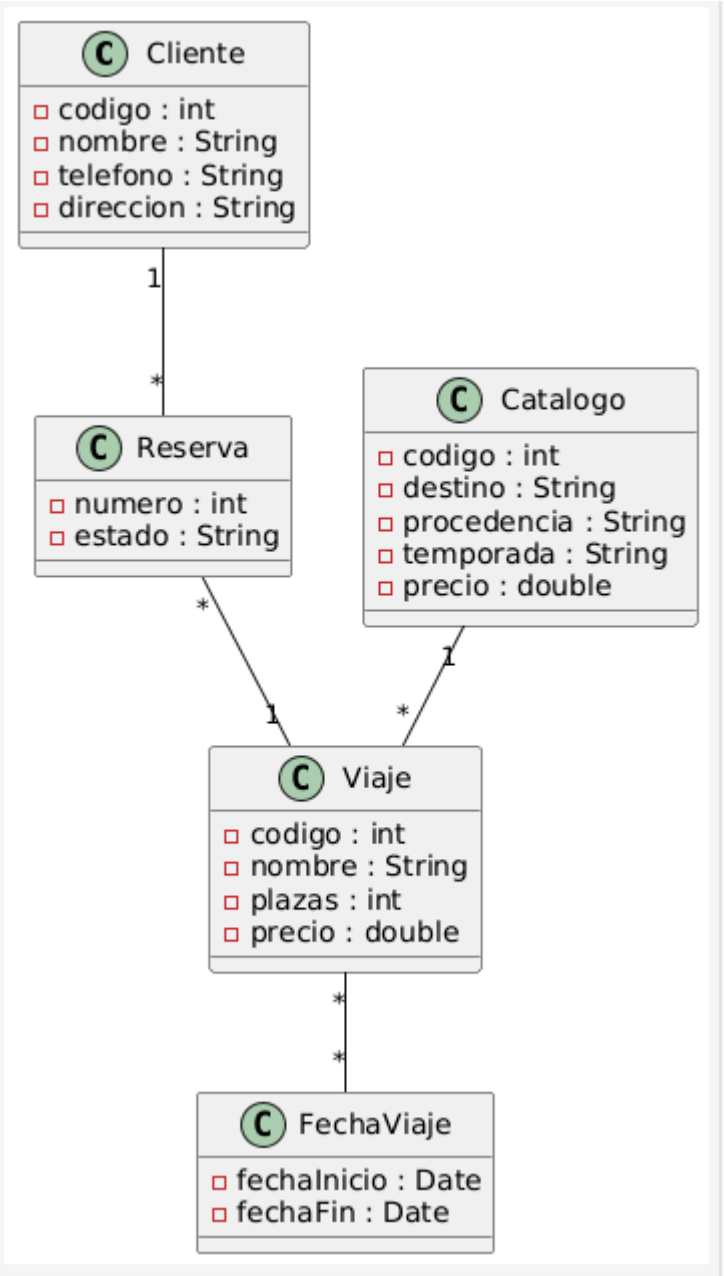
- VentanaEmpleado
- VentanaDepartamento



Ejercicio 11 – Agencia de viajes

Clases

- Cliente
- Viaje
- Reserva
- FechaViaje
- Catalogo

- 
Cliente
codigo : int
nombre : String
telefono : String
direccion : String
1
*
Reserva
numero : int
estado : String
Catalogo
codigo : int
destino : String
procedencia : String
temporada : String
precio : double
*
1
1
*
Viaje
codigo : int
nombre : String
plazas : int
precio : double
*
*
FechaViaje
fechaInicio : Date
fechaFin : Date